Attention from the ground up

Masking, scaled dot-product attention, the dot product, and multi-head mechanics

A revision reference · built from a working-through of the Transformer attention code · 22 diagrams

How to use this document. Each chapter builds from intuition to mechanics to the exact shapes. The diagrams are the load-bearing part — skim the figures first, then read the prose around any that need unpacking. Code-level pitfalls are collected in Chapter 6, and a one-page cheat sheet closes the document (Section 7).

Contents

1 Masking: hiding what attention must not see

Before attention can run, we often need to forbid certain query–key connections. Two masks do this: the **padding mask** (hide filler tokens) and the **causal mask** (hide the future). Both work the same way mechanically — they add `0` to allowed scores and `−∞` to forbidden scores *before* the softmax.

1.1 Why padding exists, and what the mask does

Transformers process sequences in batches, but real sentences differ in length. To pack them into one rectangular tensor, shorter sequences are extended with a special `<pad>` token up to the longest length in the batch. Those pad tokens carry no information — they are just filler so the shapes line up.

The danger is that attention, left unchecked, would happily attend *to* those pad positions and let their meaningless values leak into the representations of the real tokens. `create_padding_mask` builds the object that prevents this: for each sequence it produces a vector that is `0` at real positions and `−∞` at pad positions.

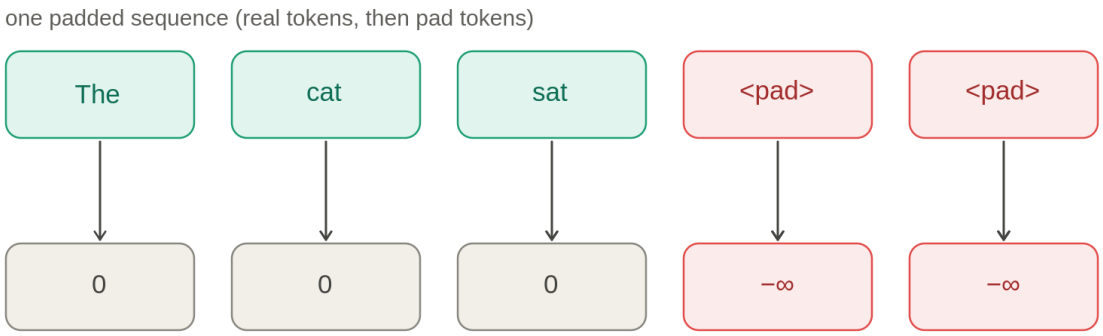

Figure 1. A padded sequence maps to a mask: 0 at real tokens, −∞ at pad tokens.

1.2 How the mask acts on attention

Attention computes a score for every (query, key) pair — a matrix where rows are query positions and columns are key positions. The mask is **added** to these scores before the softmax. Where the mask is `0`, the score is unchanged; where it is `−∞`, the score becomes `−∞`, and since softmax exponentiates, $e^{-\infty} = 0$. So pad keys end up with exactly zero attention weight: no query can pull information from them.

The mask is a single row vector over the *key* dimension, added identically to every query row — that is the broadcasting:

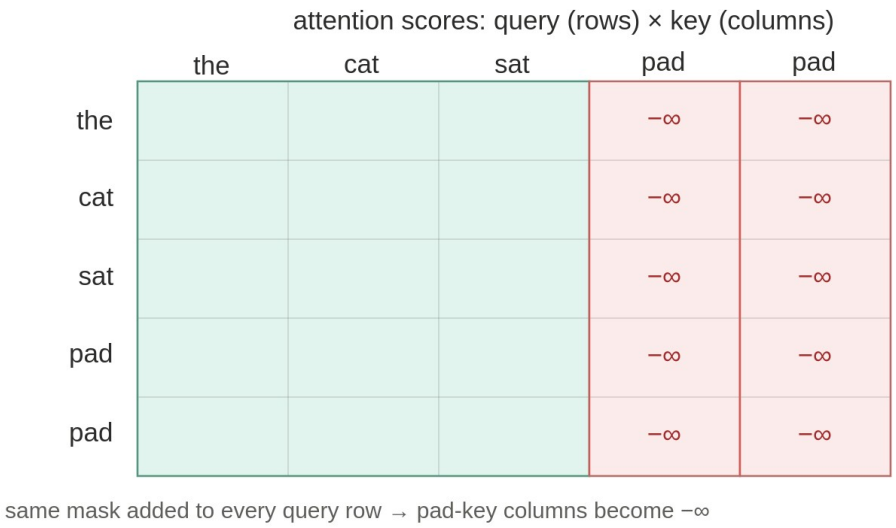

Figure 2. The padding mask zeros pad-key columns for every query row.

Key idea — mask keys, not queries. The padding mask hides pad *keys* (columns), not pad *queries* (rows). Masking a whole query row would make softmax compute `0/0 = NaN`. Pad query rows are left alone because (a) the column mask keeps real positions from ever reading them, and (b) the loss ignores pad positions via `ignore_index`.

1.3 The shape `(batch, 1, 1, seq_len)`

Attention scores live in a 4-D tensor of shape `(batch, num_heads, seq_len_q, seq_len_k)`. The mask is built to broadcast cleanly onto that:

- **batch** — one distinct mask per sequence, since each has its own pad positions.
- the first **1** broadcasts across `num_heads`: every head hides the same pad keys.
- the second **1** broadcasts across the query positions: masking depends only on *which key* you look at, not which query is looking.
- the final **seq_len** is the *key* dimension — the axis the mask actually operates on.

1.4 The causal mask: hiding the future

The causal mask is not about padding at all — it is about **time**. The decoder is trained to predict the next token. During training the whole target sequence is fed in at once for speed, which creates a cheating risk: when computing the representation at position 3, ordinary attention would let it look at positions 4, 5, 6… — the very tokens it is supposed to predict. The causal mask enforces the rule: **position `i` may attend only to positions `≤ i`** (itself and everything before).

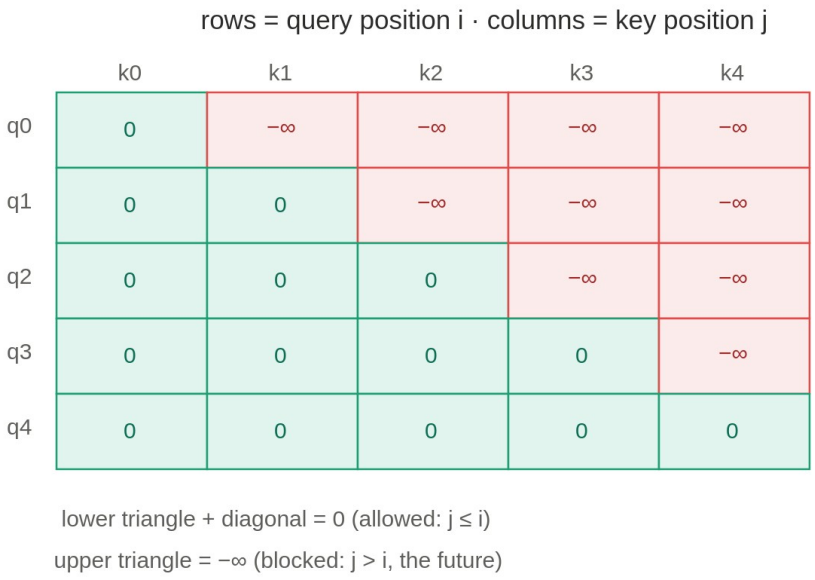

Figure 3. The causal mask: a triangle blocking the future (j > i).

Read row by row: query `q0` sees only `k0`; `q2` sees `k0,k1,k2` but not the future; the last query sees everything. The boundary is the diagonal: cell `(i, j)` is allowed exactly when `j ≤ i`. This is why the lower triangle including the diagonal is `0` and the upper triangle is `−∞`.

1.5 Padding vs causal: sizes and size conversion

Both masks end up 4-D so they can be added onto the scores, but they *start* from different natural shapes, and that drives different reshaping. The padding mask is born from the token-ID tensor `(batch, seq_len)`, so the two new size-1 axes are inserted **in the middle**. The causal mask is born from a `(seq_len, seq_len)` triangle, so the two new size-1 axes are **prepended**.

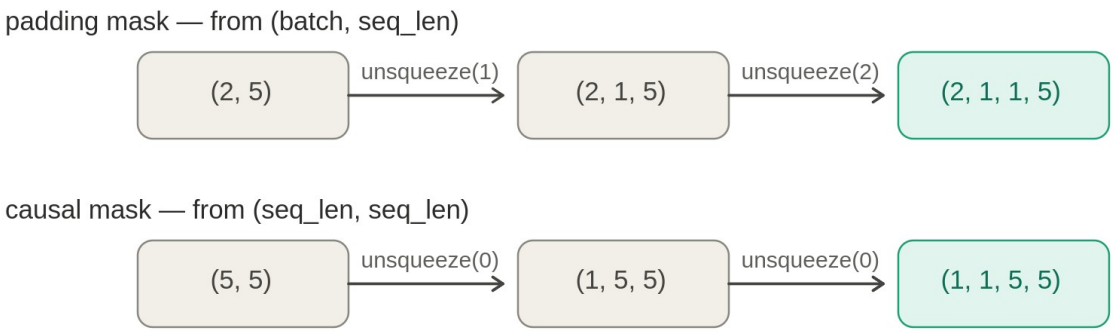

Figure 4. Padding inserts size-1 axes in the middle; causal prepends them.

Lining each final shape up against the scores `(batch, num_heads, seq_q, seq_k)`, each axis

must match or be `1` (a `1` gets stretched during the add):

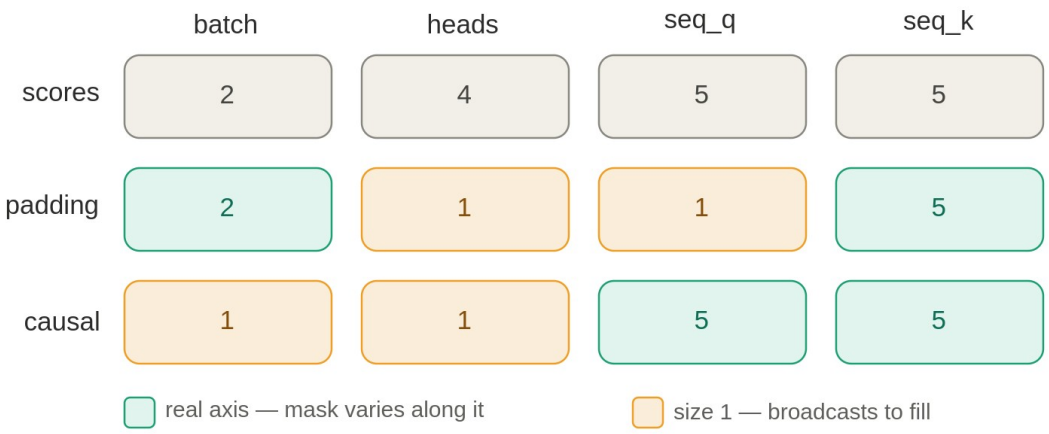

Figure 5. Axis alignment against the scores tensor: real vs broadcast.

> **The complementary pattern.** Keys: real for both. Heads: size-1 for both. The swap is in the other two axes — padding is real on *batch* but size-1 on *query*; causal is size-1 on *batch* but real on *query*. The padding mask is a per-key **vector**; the causal mask is a per-pair **square**. Because both are additive `0`/`−∞` tensors, the decoder can simply *add them together* to enforce both rules at once.

2 Scaled dot-product attention

The core operation of every Transformer: `Attention(Q, K, V) = softmax(Q K^T / √d_k) V`. It is best understood as a **soft lookup**, read inside-out.

2.1 The idea: a soft lookup

An ordinary dictionary does a *hard* lookup — one exact key returns one value. Attention does a **soft** lookup: a query is compared against *every* key, producing a relevance score for each, and the result is a blend of *all* the values weighted by relevance. Each token carries three learned vectors: a **query** (what it is looking for), a **key** (what it advertises), and a **value** (what it contributes if attended to).

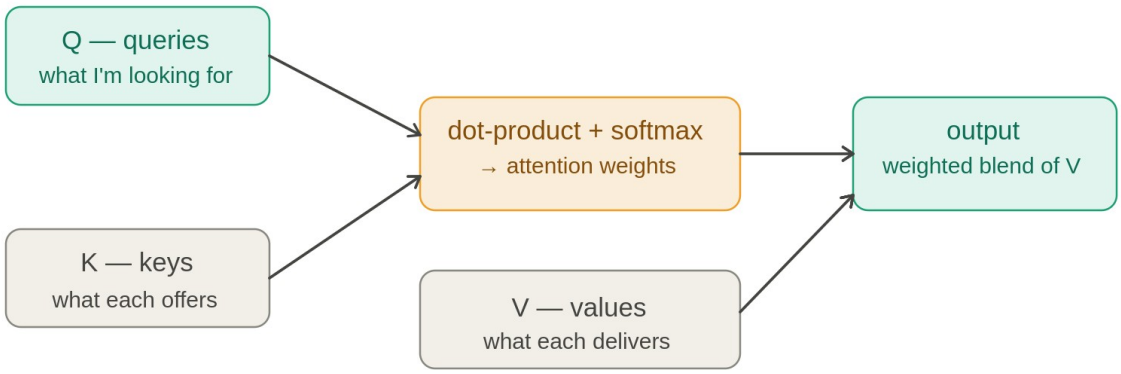

Figure 6. Attention as a soft lookup: compare queries to keys, blend values.

When Q, K, V come from the same sequence it is **self-attention**; when Q comes from one sequence and K, V from another (decoder querying the encoder) it is **cross-attention** — which is why the signature allows `seq_len_q` to differ from `seq_len_k`. Always `seq_len_k == seq_len_v`, since every key has exactly one value.

2.2 The pipeline, with shapes

Two *shape-changing* matrix multiplies bracket four *shape-preserving* steps in the middle:

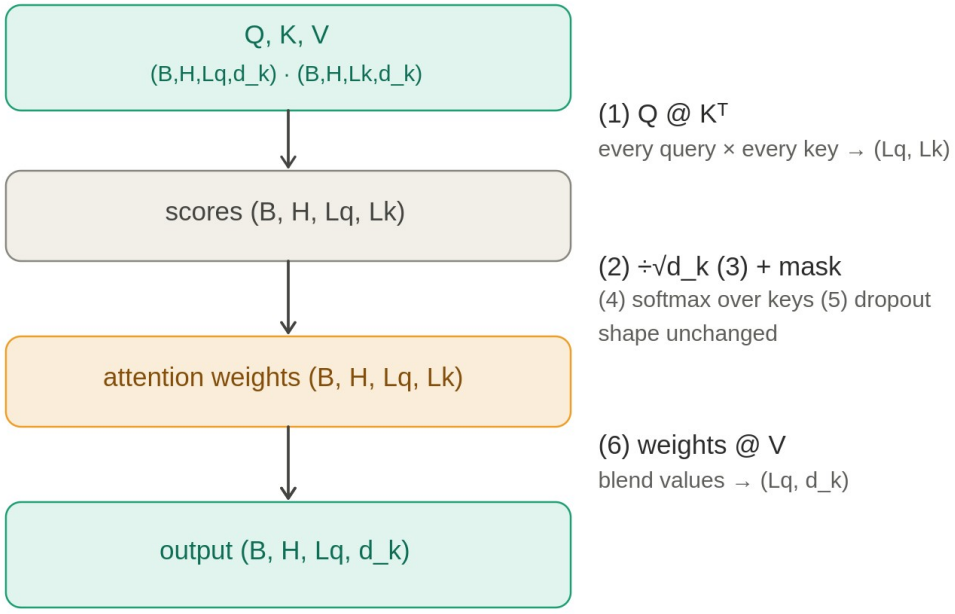

Figure 7. The attention pipeline and the shape at every step.

The function returns two things: the `output` (blended values, fed onward) and the `attn_weights` (the `(Lq, Lk)` distribution, returned for visualization and debugging since it is otherwise discarded by the final matmul).

2.3 Step 1: `Q Kᵀ` — comparing every query to every key

The dot product is the similarity measure: two vectors pointing the same way have a large positive dot product; perpendicular vectors give zero. So `q·k` answers "how relevant is key j to query i?" Doing this for every pair fills a `(Lq, Lk)` grid. The shape mechanics are a standard matmul where the shared inner dimension `d_k` is summed over and disappears:

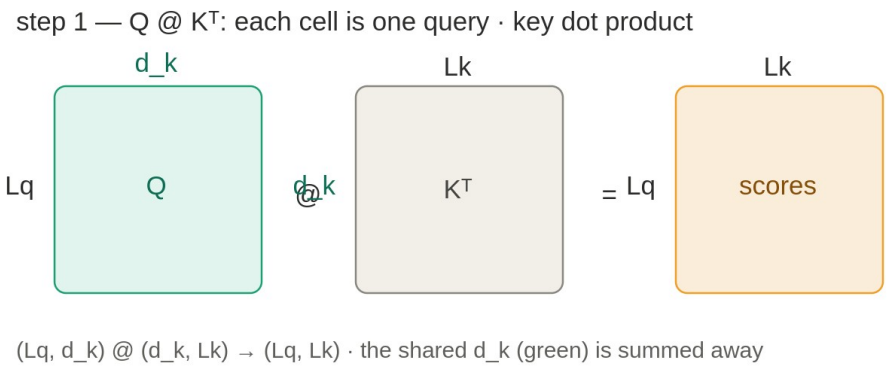

Figure 8. Q K^T contracts the shared d_k to produce the score grid.

The K^T (transpose of the last two axes) lines the `d_k` columns of Q up against the `d_k` of K so they can be dotted. The full 4-D version rides the `batch` and `num_heads` axes along untouched.

2.4 Step 2: `/ √d_k` — taming the scale

If the `d_k` components of a query and key are roughly independent with mean 0 and variance 1, their dot product is a sum of `d_k` terms each with variance ≈ 1. Variances add, so the dot product has variance ≈ `d_k` — its typical magnitude grows like `√d_k`. For `d_k = 64` that is a spread of ±8; for `d_k = 512`, about ±23.

Large magnitude is a problem because softmax responds to the *absolute* gaps between inputs. When scores are spread far apart, softmax saturates — nearly all weight piles onto one key, the rest go to ~0, and the gradient there nearly vanishes, stalling learning. Dividing by `√d_k` rescales to variance ≈ 1 regardless of `d_k`:

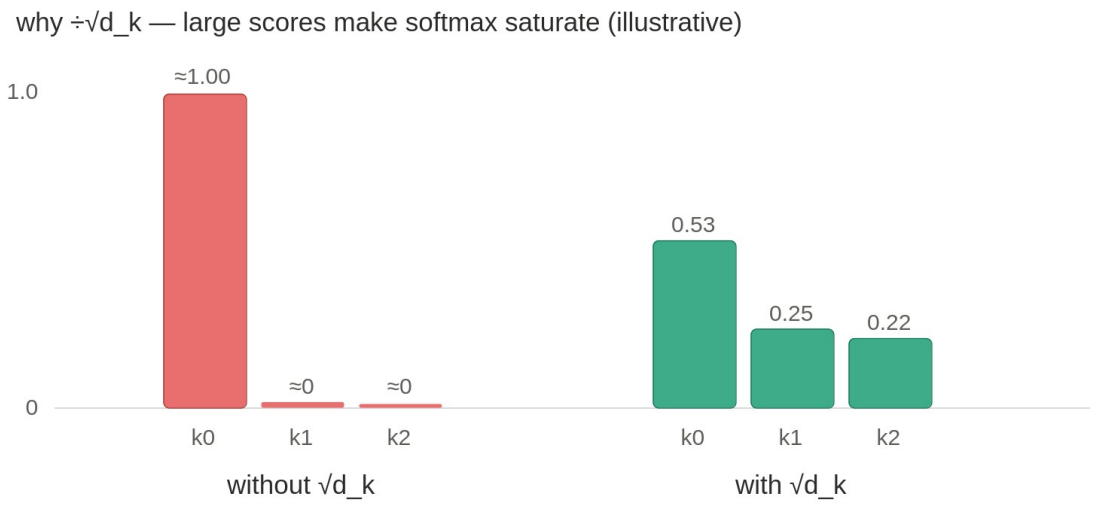

Figure 9. Large scores saturate softmax; √d_k restores a balanced distribution.

This is not cosmetic. Without the `√d_k`, a large head dimension pushes the softmax into a flat, near-one-hot regime where gradients vanish and the model cannot train.

2.5 Steps 3–4: mask, then softmax

The mask is *added* to the scaled scores (`0` leaves a score alone, `−∞` forces it to vanish). Then softmax runs over the last axis — the keys — turning each query's row of scores into a probability distribution: every weight `≥ 0`, the row sums to `1`. Masked positions, being `−∞` going in, come out as exactly `0`. The result is `attn_weights`, shape `(B, H, Lq, Lk)`. Dropout (step 5) is optional training-time regularization on those weights; at inference it is a no-op.

2.6 Step 6: `weights V` — the blend

Each query's output is the weighted sum of all value vectors, mixed by that query's attention distribution. This second matmul, `(Lq, Lk) (Lk, d_k) → (Lq, d_k)`, contracts the *key* axis `Lk`:

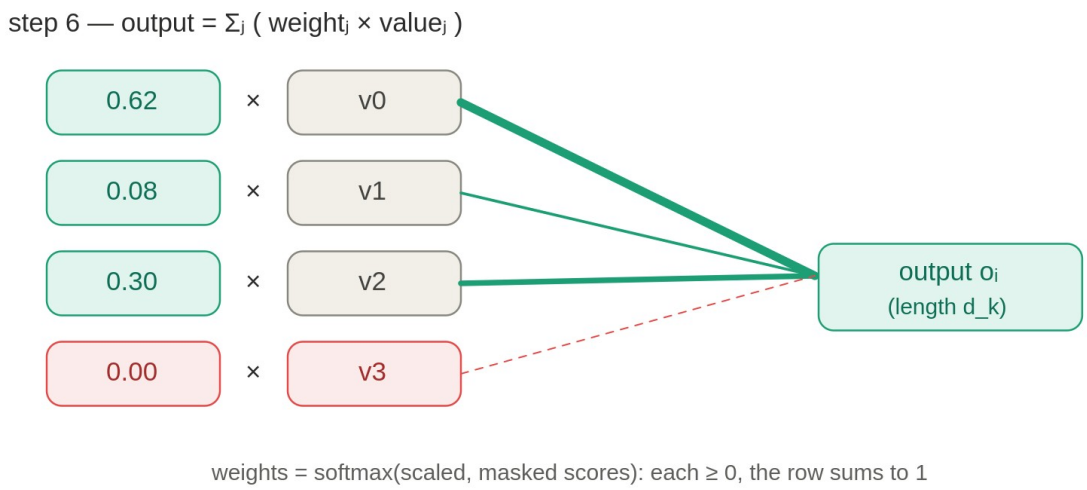

Figure 10. The output is a relevance-weighted blend of value vectors.

The masked value (weight 0) is dropped from the blend — the entire payoff of the masking pipeline. The output lands at `(B, H, Lq, d_k)`: one `d_k`-vector per query, the same shape the queries arrived in, now carrying context.

The symmetry worth remembering. The first matmul sums over `d_k` to *compare* (turning `d_k` into `Lk`); the second sums over `Lk` to *aggregate* (turning `Lk` back into `d_k`). The `d_k` axis disappears then returns; the `Lk` axis appears then disappears.

3 The dot product: why $\Sigma\ a \cdot b = |a||b|\cos\theta$

The dot product has two faces — the component sum $a_1b_1 + a_2b_2 + \ldots$ (what the matmul computes) and the geometric $|a||b|\cos\theta$. They are the same number, and that equivalence is what makes the dot product a similarity measure.

3.1 The geometric meaning of alignment

The dot product of two vectors is $|a||b|\cos\theta$, where θ is the angle between them. Aligned vectors give a large positive value; perpendicular vectors give zero; opposing vectors give a negative value. Since `W_Q` and `W_K` are learned, the model learns to point a query and a relevant key in the same direction, making their dot product large:

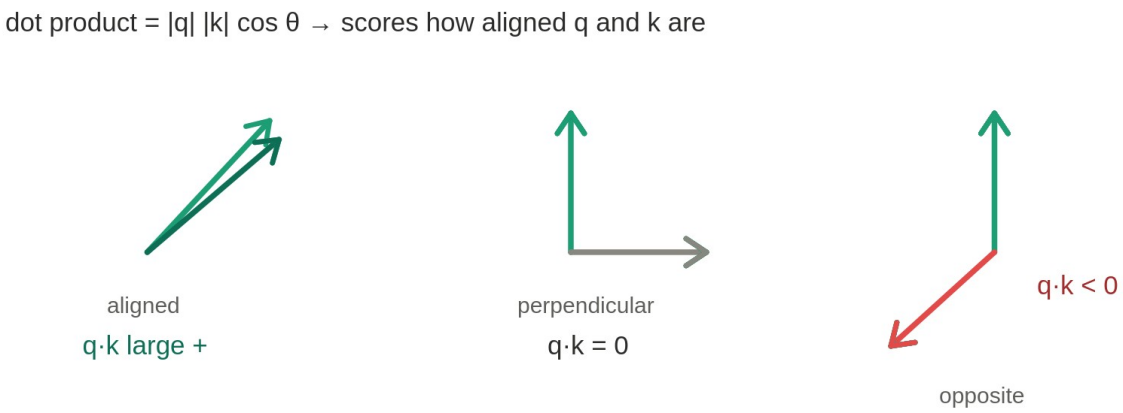

Figure 11. The dot product measures alignment between vectors.

3.2 The shadow (projection) picture

$|b|\cos\theta$ is the length of **b's shadow** cast onto a's direction. So $|a||b|\cos\theta$ = (length of a) × (length of b's shadow). When a lies on the x-axis it is $(|a|, 0)$, so the component sum collapses to $|a| \cdot b_1$ — and b_1 (b's x-coordinate) *is* its shadow, which equals $|b|\cos\theta$:

a along the x-axis: component sum = |a| × b's shadow

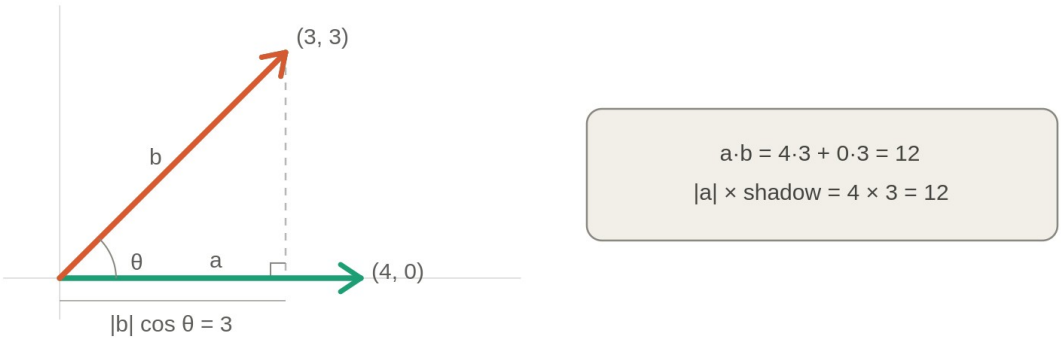

Figure 12. |b|cosθ is b's shadow on a's direction.

3.3 The general proof (law of cosines)

For any orientation, form the triangle of the two vectors and compute the length of the third side, `b − a`, two ways:

the third side is the vector b − a

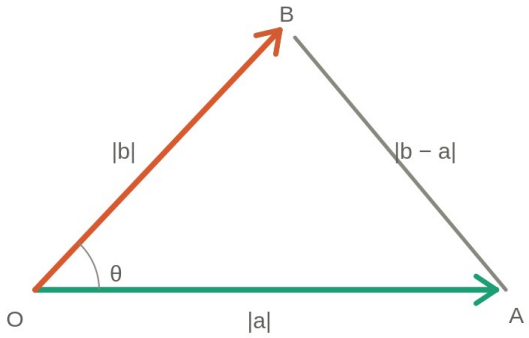

Figure 13. The law-of-cosines triangle used to prove the identity.

From **geometry** (law of cosines): `|b−a|² = |a|² + |b|² − 2|a||b|cosθ`.

From **algebra** (squared length = sum of squared components): `|b−a|² = Σ(bᵢ−aᵢ)² = |b|² − 2(a·b) + |a|²`, where `a·b = Σaᵢbᵢ`.

Setting the two equal and cancelling `|a|²` and `|b|²` leaves `a·b = |a||b|cosθ`. It is just Pythagoras generalized: at `θ = 90°`, `cosθ = 0`, so perpendicular vectors have zero dot product and the identity collapses to the Pythagorean theorem.

> **Tie to attention.** Each entry of $Q\ K^T$ is a row·column = component sum = $|q||k|\cos\theta$. So each attention score literally asks “how aligned is this query with this key?” A relevant key earns a large dot product and high attention; an orthogonal one scores near zero; an opposing one scores negative.

4 Multi-head attention

Instead of one attention over the full `d_model` space, run `num_heads` independent attentions over `d_k`-dimensional subspaces, then concatenate and mix. Five stages: project, split, attend, merge, project.

4.1 Where Q, K, V come from

The attention function *receives* Q, K, V already formed; to understand their sizes you must see them being born. Every token starts as an embedding vector of length `d_model`. A sentence of `L` tokens is an `(L, d_model)` matrix:

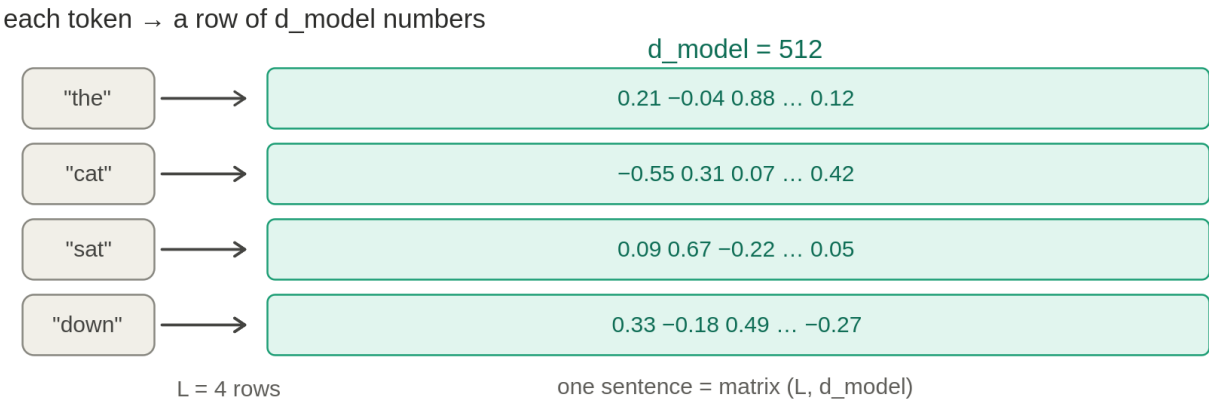

Figure 17. Each token becomes a row of d_model numbers.

Each of Q, K, V is the embedding matrix multiplied by a learned weight matrix. A weight matrix of shape `(d_model, d_k)` projects each token's `d_model`-vector down to a `d_k`-vector. There are three such matrices — `W_Q, W_K, W_V` — and they are what the model learns:

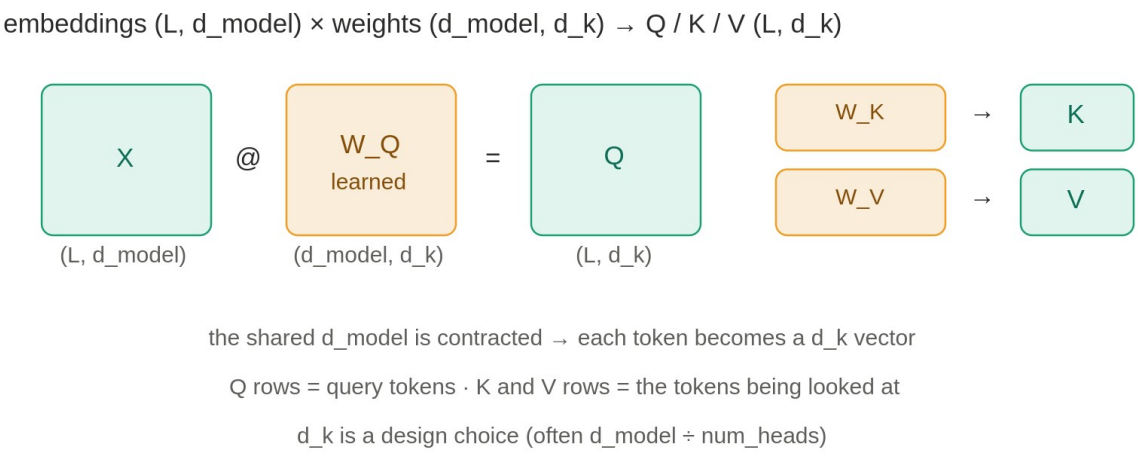

Figure 18. Q, K, V are produced by projecting embeddings with learned matrices.

This explains every size. `d_k` is the per-head width, usually `d_model ÷ num_heads`. Q and K must share `d_k` because step 1 dots them. `seq_len_q` and `seq_len_k` are just the row counts of Q and K — equal in self-attention, possibly different in cross-attention.

4.2 The forward pass, end to end

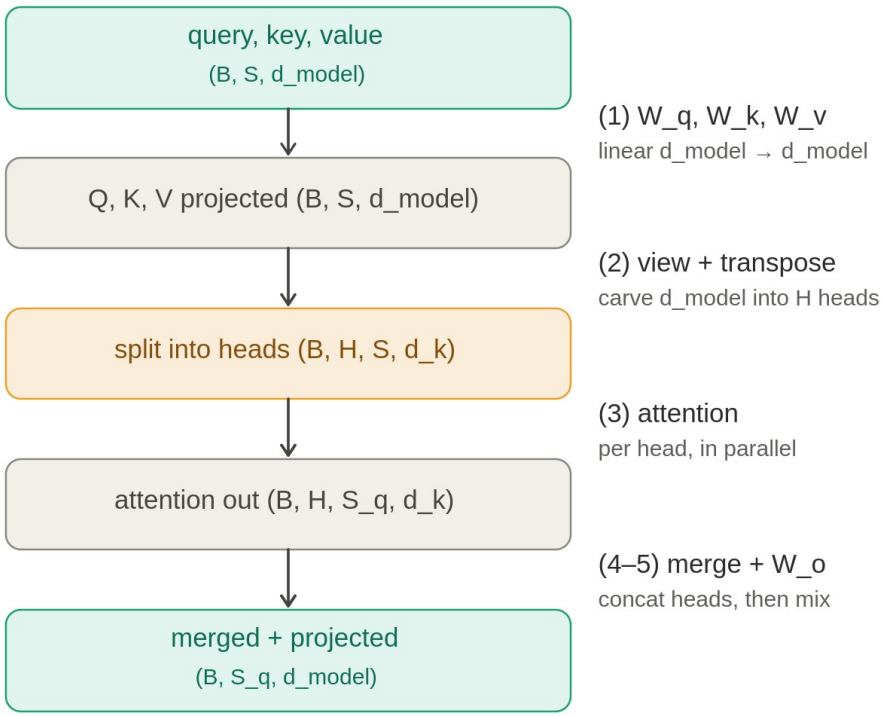

Figure 14. Multi-head attention: project, split, attend, merge, project.

Step 1: the projections

`Q = self.W_q(query)` applies an `nn.Linear(d_model, d_model)`. The crucial point: a *single* `(d_model, d_model)` matrix computes all heads' projections at once — the per-head split happens later by reshaping. That is the efficiency win over keeping `num_heads` separate `(d_model, d_k)` matrices: same math, one fused matmul.

Step 2: split into heads

`Q.view(B, -1, H, d_k).transpose(1, 2)`. The `view` reinterprets the contiguous `d_model` axis as `(H, d_k)` — no data moves, the `-1` infers `S`. The `transpose` pulls heads up to a batch-like position so attention runs independently per head:

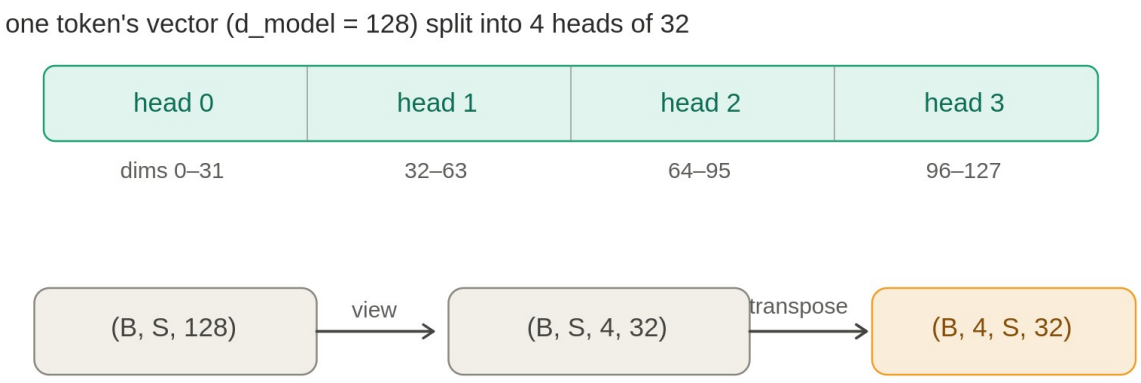

Figure 15. Splitting one d_model vector into H heads via view + transpose.

Step 4: merge heads (the exact inverse)

`attn_out.transpose(1, 2).reshape(B, -1, d_model)`. The transpose restores the `(…, S_q, H, d_k)` ordering, then reshape glues `(H, d_k)` back into one `d_model` axis — concatenating the heads:

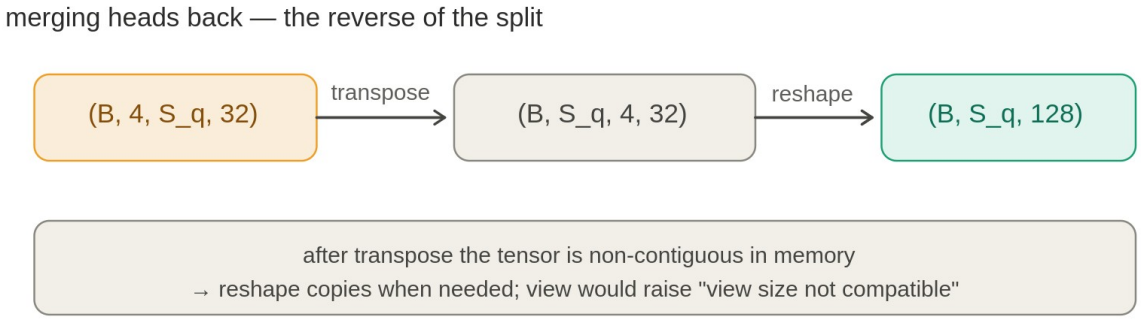

Figure 16. Merging heads back via transpose + reshape (the exact inverse).

Order matters: transpose *then* reshape. `reshape` (not `view`) is required because `transpose` leaves the tensor non-contiguous. And the transpose must come first — reshaping `(B, H, S_q, d_k)` directly would glue the head axis to the sequence axis and silently scramble the output with no error.

4.3 Step 5: `W_o` and why the heads need it

After the merge, the four heads are just *stacked* side by side — dims 0–31 are purely head 0's work, 32–63 purely head 1's, and so on. The concatenation glued them together but did not *combine* them. `W_o` is the one operation that lets information cross between heads:

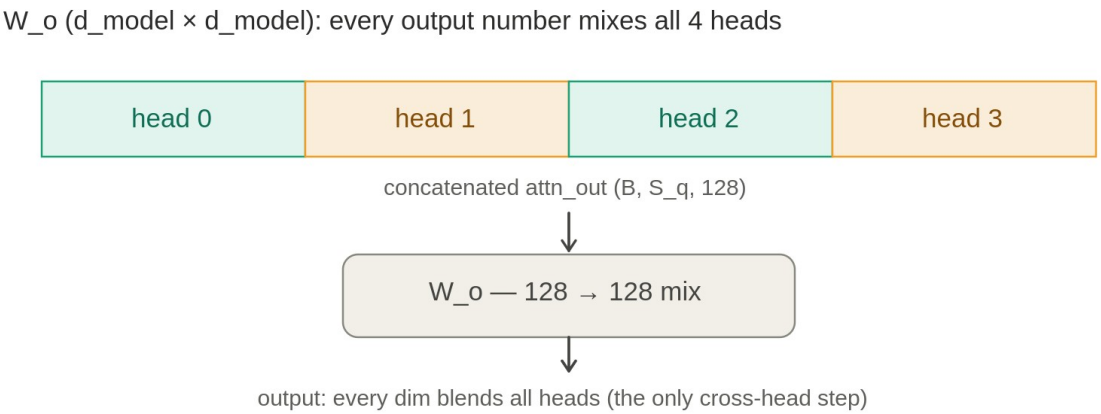

Figure 21. W_o is the only step where information crosses between heads.

Because `W_o` is a full `(d_model, d_model)` matrix, every one of its output numbers is a weighted sum of *all* inputs — so every output draws on all four heads at once. That is the only place in the entire attention module where heads coordinate.

5 Linear layers and the matrix × vector multiply

Every projection (`W_q, W_k, W_v, W_o`) is one operation: `nn.Linear` running a matrix–vector multiply on each token. Understanding it explains how shapes are preserved and why widths are what they are.

5.1 What `nn.Linear` does to one vector

`self.W_q = nn.Linear(d_model, d_model)` holds a learned weight matrix `W` of shape `(d_model, d_model)` and a bias `b`. Calling `self.W_q(query)` runs `query @ W^T + b` — a matrix multiply plus a bias add. For a single token, a `d_model`-vector goes in and a *different* `d_model`-vector comes out, where every output number is a learned weighted mix of all input numbers:

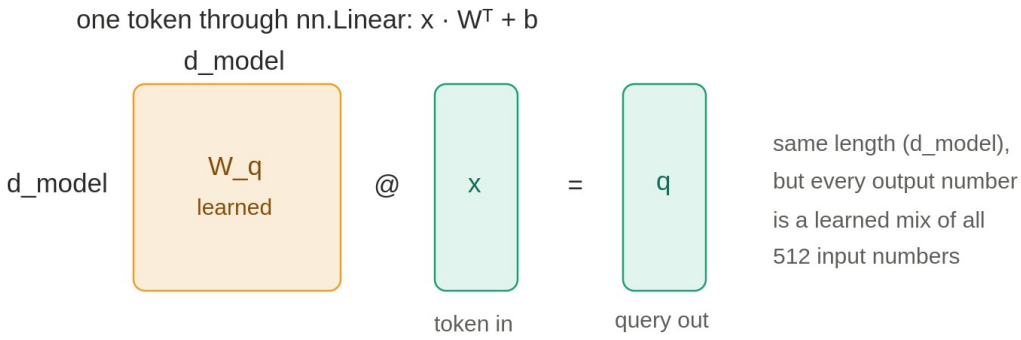

Figure 19. nn.Linear on one token: a learned mix into a same-length vector.

5.2 Why a 3-D tensor works unchanged

`nn.Linear` transforms only the **last** axis. It treats `B` and `S` as a stack of independent token-vectors and applies the *same* `W` to each. So `(B, S, d_model) → (B, S, d_model)`: only the contents of each token's vector change; the sentence-and-token scaffolding is untouched. This is the same “the operation only touches the last axis, everything else is a parallel stack” idea that gives attention its batch and head axes.

5.3 Three roles from one token

The same input token is pushed through three *different* learned matrices, producing three different

vectors. `W_q` learned to make a good “what am I looking for” vector, `W_k` a “what do I offer,” `W_v` a “what will I pass along”:

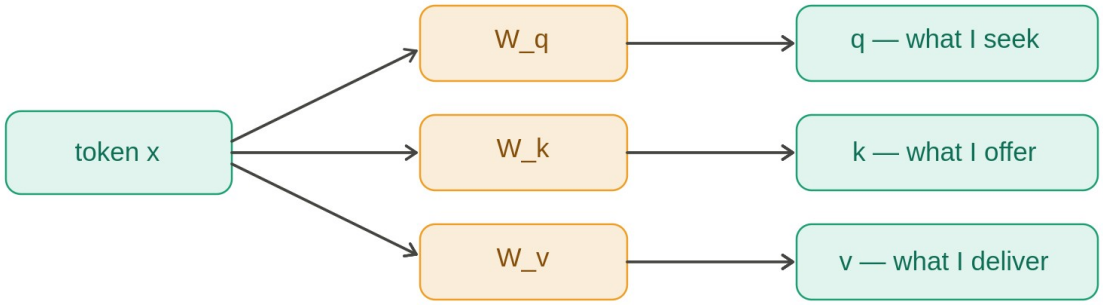

Figure 20. One token, three learned matrices, three roles.

This is why the module takes three inputs. Self-attention: `attn(x, x, x)`. Cross-attention: `attn(decoder_x, encoder_x, encoder_x)` — the decoder asks the questions, the encoder supplies what is matched and retrieved.

5.4 First principles: matrix × vector, and why width is preserved

The rule of a matrix–vector multiply: **each output number is one row of the matrix, dotted with the whole input.** A small 3×3 example makes both facts visible:

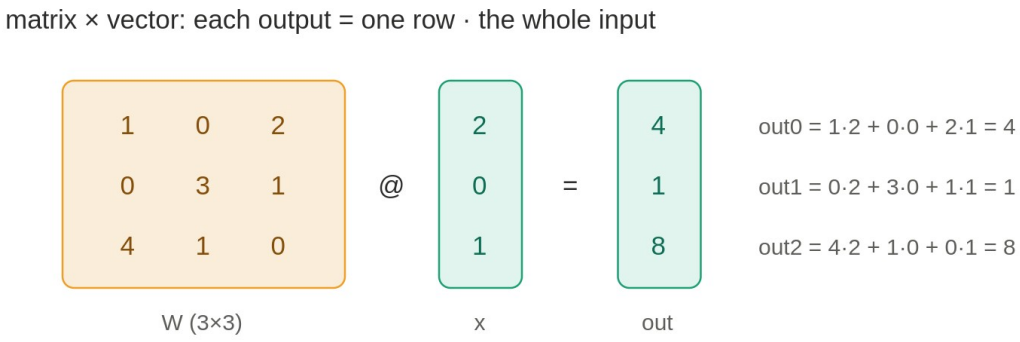

Figure 22. Matrix × vector: each output is one row dotted with the whole input.

Two observations carry the whole story. First, every output used *all* input numbers (each dot product ran across the entire input). Second, the number of outputs equals the number of **rows** — one dot product per row.

Why `W_o` being square preserves the shape. A matrix's shape reads as `(rows, columns) =`

`(outputs, inputs)`. `W_o` is `128×128`: 128 rows give 128 outputs, 128 columns accept a 128-vector. So a 128-vector goes in, a 128-vector comes out — the width is preserved *because the matrix is square*. And because each of the 128 outputs dots across all 128 inputs (= the four heads concatenated), every output blends all four heads. The shape is preserved by the row count; the mixing is performed by the dot products. The outer `(B, S_q)` structure is untouched because this `128→128` multiply runs once per token, never across tokens.

6 Implementation notes and pitfalls

Code-level details that matter in practice — numerical safety, device/dtype, the two-value return, and Python's reference semantics.

6.1 `-inf` vs `torch.finfo(dtype).min`

Filling masked positions with `float('-inf')` works on normal inputs but has a sharp edge: if an entire query row becomes `-inf` (a fully-padded sequence, or a causal row combined with padding so every allowed key is also pad), softmax computes `0/0 = NaN`, which then propagates silently through the network and poisons the loss. Using `torch.finfo(dtype).min` — the most-negative *finite* value for the dtype — avoids this: finite-but-equal values subtract to 0 and softmax falls back to a harmless uniform row.

Why not `-1e9`? In `fp16`, `-1e9` overflows to `-inf` (max magnitude ~65504), quietly reintroducing the NaN risk. `finfo(dtype).min` is by definition always representable. Caveat: summing two masks that both use `finfo.min` can overflow to `-inf`; if you combine additive masks heavily, OR them as booleans and convert to additive once.

6.2 Device and dtype

The padding mask gets its device for free via `zeros_like(seq)`. The causal mask, built from a bare `int` with no tensor to inherit from, defaults to CPU — you **must** pass `device` explicitly or you will hit a “tensors on different devices” error when adding it to GPU scores. Likewise, hard-coding `float32` forces an upcast when scores are `bf16`/`fp16`; make the mask dtype match.

6.3 The mask convention is an additive-float footgun

`scores = scores + mask` only works for `0`/`-inf` float masks. A boolean mask (where `True` means “ignore,” as in `nn.MultiheadAttention`) passed here will not error — it silently adds `1.0` at every `True` position and corrupts the scores. Either assert the dtype or handle both:

```
if mask is not None:

    if mask.dtype == torch.bool:
```

```
        scores = scores.masked_fill(mask, float("-inf"))
    else:
        scores = scores + mask
```

6.4 Why two values are returned

The function returns `(output, attn_weights)`. The `output` `(B, H, S_q, d_k)` is the blended values the network consumes — the forward pass needs only this. The `attn_weights` `(B, H, S_q, S_k)` is the softmax distribution *before* it is collapsed against V; once `weights @ V` runs, that grid is gone, so it must be surfaced here. It is the only interpretable part of attention (the "attention maps"), useful for visualization and for checking that masks zeroed the right positions. Callers typically write `out, _ = ...` when they do not need it.

> **Dropout/return subtlety.** If `attn_weights = dropout(attn_weights)` runs before the return, the weights you get back during training are the perforated, rescaled ones — not the clean softmax the docstring promises. To plot clean attention maps, capture the pre-dropout weights (or inspect in `eval()` mode).

6.5 Python return semantics: pass by object reference

Python is neither C++ pass-by-value nor pass-by-reference; it is **pass by object reference** ("call by sharing"). `return output, attn_weights` builds a tuple of *references* — no tensor data is copied. The caller's names bind to the same tensor objects. So an in-place mutation (`out.add_(1)`, trailing underscore) is seen by every reference; reassignment (`out = out + 1`) makes a new tensor and leaves the original untouched. The function can never rebind the caller's *name* — it only returns objects. If you need independent data, return `output.clone()`; the default `return` never clones.

6.6 Performance: the fused kernel tradeoff

The biggest speed lever is `F.scaled_dot_product_attention` (PyTorch 2.x), which dispatches to FlashAttention / memory-efficient kernels and never materializes the full `(Lq, Lk)` scores matrix. The catch: those kernels *do not return attention weights* — not materializing them is the point. A `need_weights` flag (as in `nn.MultiheadAttention`) is the usual way to have both: fused kernel when weights aren't needed, explicit path when they are. For a from-scratch learning implementation,

the explicit version is the right call.

---

7 One-page summary

Concept	The one-line takeaway
Padding mask	Per-key vector `(B,1,1,S)`; `-inf` at pad keys so no query reads them.
Causal mask	Per-pair square `(1,1,S,S)`; upper triangle `-inf` so no token sees the future.
Mask mechanism	Add to scores before softmax; $e^{-inf}=0$ gives zero weight.
Q K^T	Compare; sums over `d_k`; produces `(Lq, Lk)` similarity grid.
√d_k scaling	Keeps dot-product variance ~1 so softmax doesn't saturate.
softmax	Per query row over keys; non-negative, sums to 1.
weights V	Aggregate; sums over `Lk`; restores `(Lq, d_k)`.
dot product	$\Sigma\ a\cdot b = \|a\|\|b\|\cos\theta$ — a signed alignment score (law of cosines).
Multi-head	Split `d_model` into `H×d_k`, attend per head, concat, mix with `W_o`.
Linear layer	Per-token $x\ W^T+b$; touches last axis only; square keeps width.
W_o	The only cross-head step; square `(d_model,d_model)` blends all heads.
Two-value return	`output` feeds the network; `attn_weights` is for inspection.
NaN safety	Prefer `finfo(dtype).min` over `-inf`/`-1e9` for masked fills.